Documentation (Summit)

Contents

Setup Information and Installation Instructions 1

Development and Maintenance Guide 3
- Development Environment and Continuous Integration Framework . . . 3
- Set Up Development Environment . . . 4
- Build Tasks . . . 4
- Programming Conventions . . . 5
- Configuration Management . . . 5
- Version Numbering . . . 5
- Deployment . . . 5
- Release Process . . . 12
- Planned Changes . . . 12
- Developer Troubleshooting . . . 12

Setup Information and Installation Instructions

This section describes how to install the game and lists prerequisites that must be installed before installing the game.

Prerequisites

Before installing the game, ensure the following are installed:

- Java 11 (https://www.oracle.com/java/technologies/javase/jdk11-archive-downloads.html)

Installation

This section includes a summary of the installation instructions and a more detailed section with screenshots.

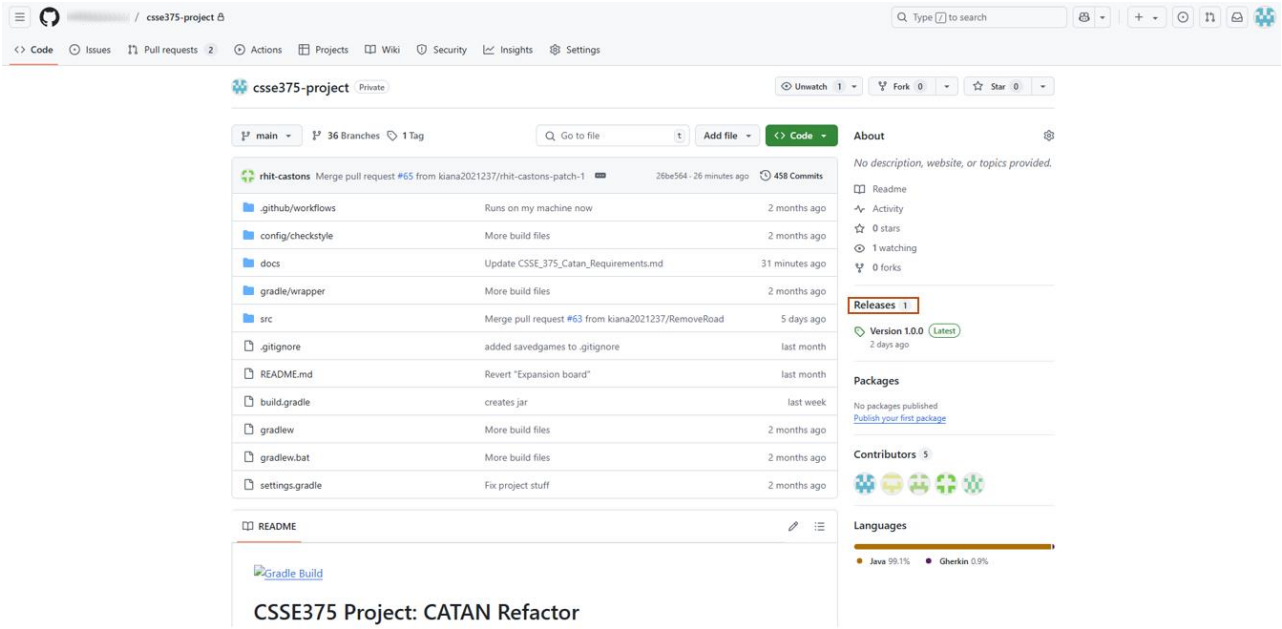
Summary

To install the game,

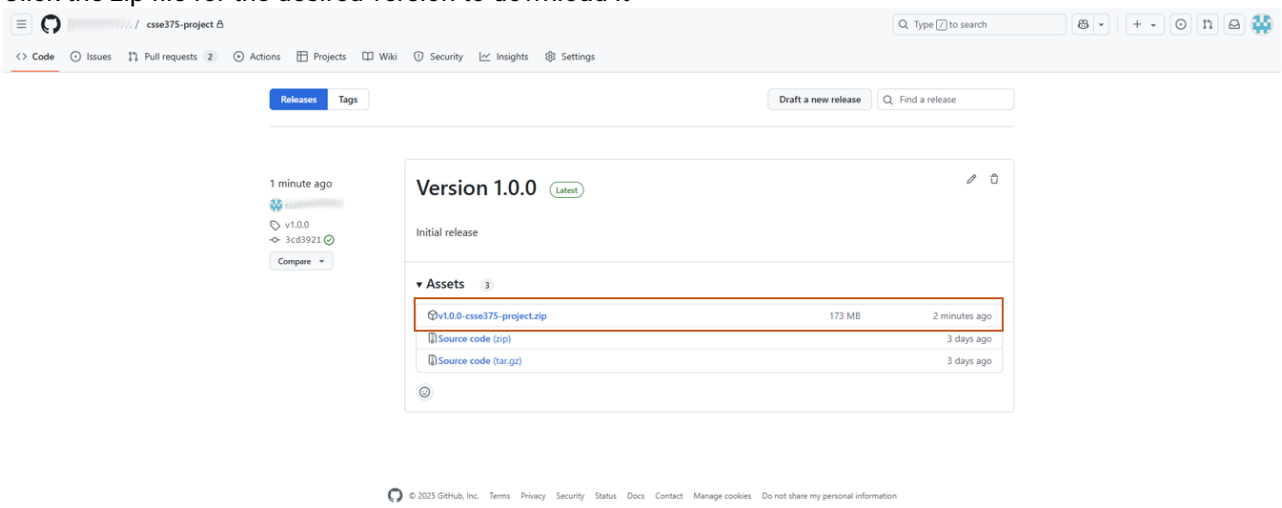
1. Go to https://github.com/summit2021237/csse375-project/releases
2. Download and extract the zip file for a release
3. Run the JAR file (`csse375-project-1.0.jar`) in the zip file to run the game
   (a) Note: There should be a `src/main/resources` directory that was extracted with the JAR file

Instructions with Screenshots

1. Go to https://github.com/summit2021237/csse375-project/releases
   (a) Or from https://github.com/summit2021237/csse375-project, click on “Releases” in the right sidebar

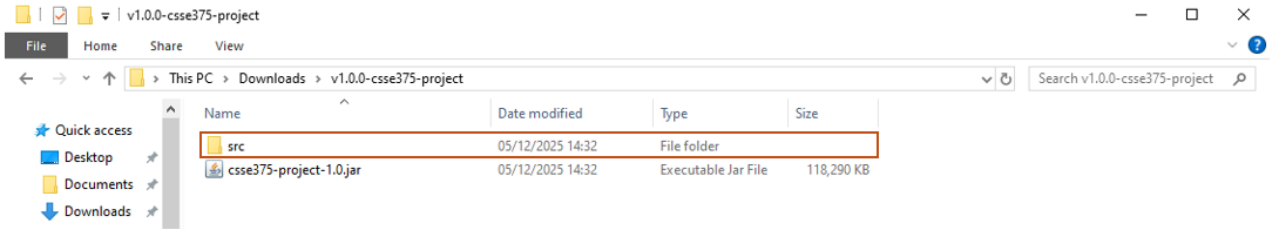


2. Click the zip file for the desired version to download it

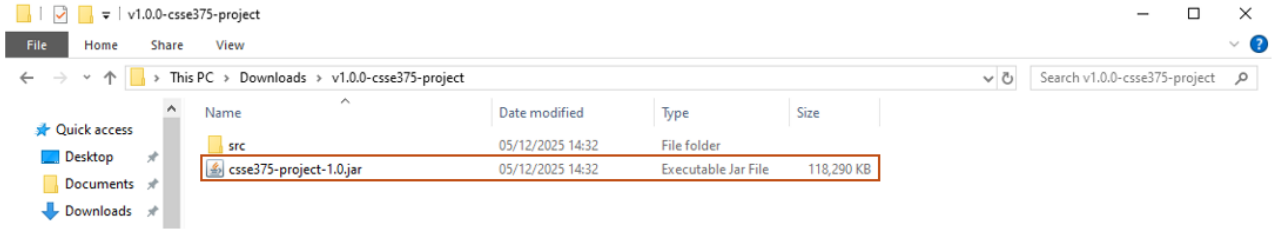


3. Extract the files from the zip file

4. Go to the directory that the files were extracted in
   (a) Note: There should be an `src/main/resources` directory that was extracted with the JAR file



5. Double click the JAR file (`csse375-project-1.0.jar`) to run the game



Development and Maintenance Guide

This guide describes the development environment, programming conventions, configuration management, release process, planned changes, and troubleshooting procedures. It is intended for developers, maintainers, and helpdesk staff.

We provide screenshots of installation, configuration, and deployment using IntelliJ, but the user can use any IDE.

Development Environment and Continuous Integration Framework

Catan has a GitHub repository for tracking versions and Gradle as the build tool. The repository has a continuous integration framework using GitHub Actions to run a Gradle build on three different operating systems, ensuring that all the unit tests, integration tests, regression tests, styling checks, and mutation checks pass when creating a pull request to merge into main.

Overview

Repository: https://github.com/summit2021237/csse375-project
Java: JDK 11
JUnit: 5.8.1
Gradle: 7.4

File Structure

`csse375-project/build`	Outputs from Gradle tasks
`csse375-project/config`	Tool configuration files
`csse375-project/docs`	Documentation
`csse375-project/src/main/java`	Source code
`csse375-project/src/main/resources`	Game resources
`csse375-project/src/test/java`	Test files
`csse375-project/src/test/resources`	Test resources

Tools

Gradle: We use Gradle because that was the build tool that most members of the team were familiar with. The original copy of the game was also set up with Gradle and had tasks for testing and style checks. Gradle has a built-in jar task, which can be configured to create a JAR for release. The build.gradle file is in the top-level directory.

JUnit: JUnit is a testing framework that lets developers easily write tests in Java to test Java code.

Checkstyle: Checkstyle runs quickly and ensures the formatting in our codebase is consistent for readability and understandability.

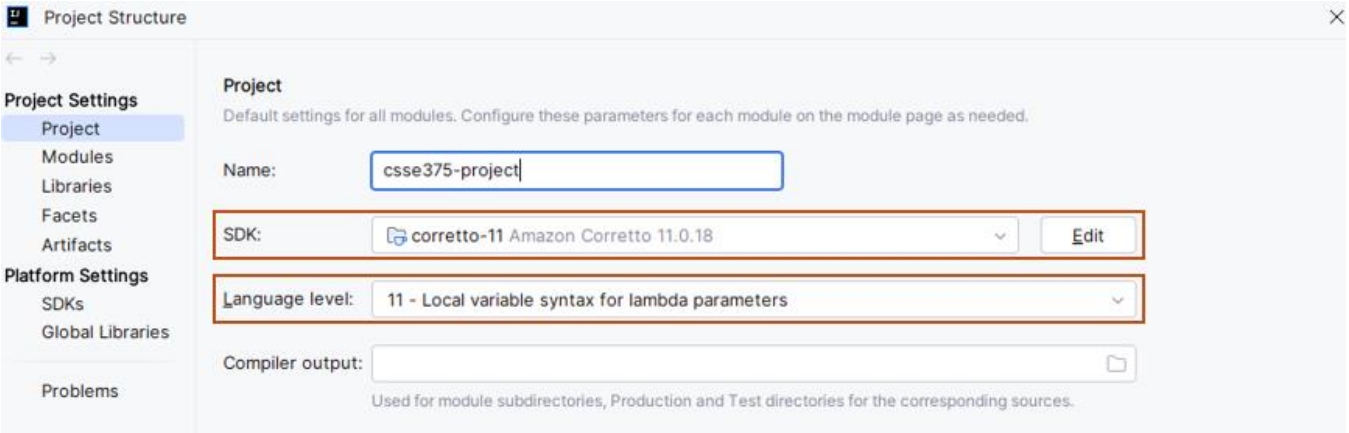
Set Up Development Environment

To set up the development environment,

1. Clone the repository (https://github.com/summit2021237/csse375-project)
2. Open the project in an IDE and configure the project and Gradle

With IntelliJ,

1. IntelliJ will automatically detect that the project uses Gradle and start a build
2. Navigate to the Project Structure (under the File menu)
3. Check that the project uses Java 11



Build Tasks

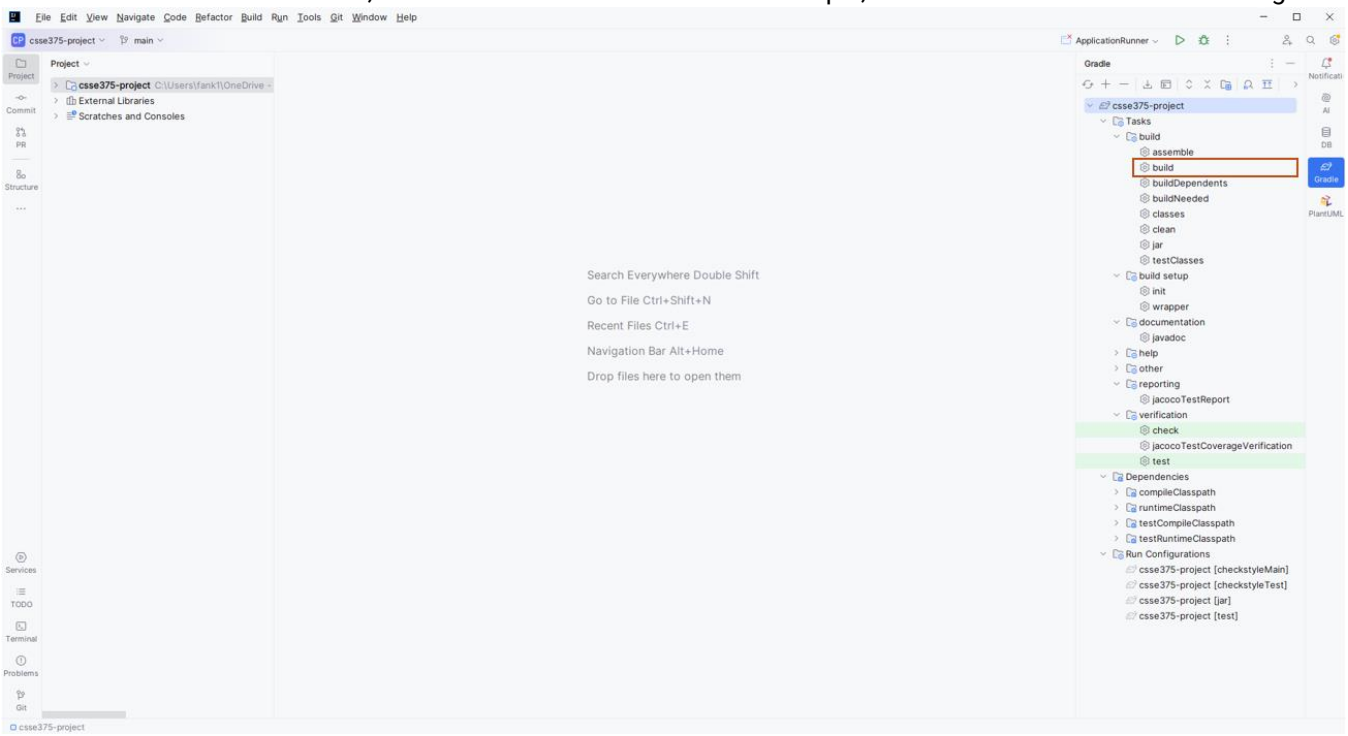
The following table describes the implemented Gradle build tasks. The most often used tasks are

- `build/build`
- `verification/test`

Task	Description
`build/assemble`	Assembles the game
`build/build`	Assembles the game and runs the tests
`build/clean`	Removes the build directory
`other/checkstyleMain`	Runs checkstyle on the production code
`other/checkstyleTest`	Runs checkstyle on the test code
`other/codeCoverageInfo`	Displays summary of code coverage results from jacoco
`reporting/jacocoTestReport`	Generates a code coverage report in the build directory
`verification/check`	Runs all style checks and mutation, unit, integration, and regression tests
`verification/test`	Runs all JUnit tests

Running a Build Task

In IntelliJ's Gradle Tool Window, double click the task to run. For example, double click this item to build the game.



Programming Conventions

(Written by someone else)

Configuration Management

(Written by someone else)

Version Numbering

This project uses standard version number conventions, where X.Y.Z is the version with

- X: major version number
- Y: minor version number
- Z: patch number

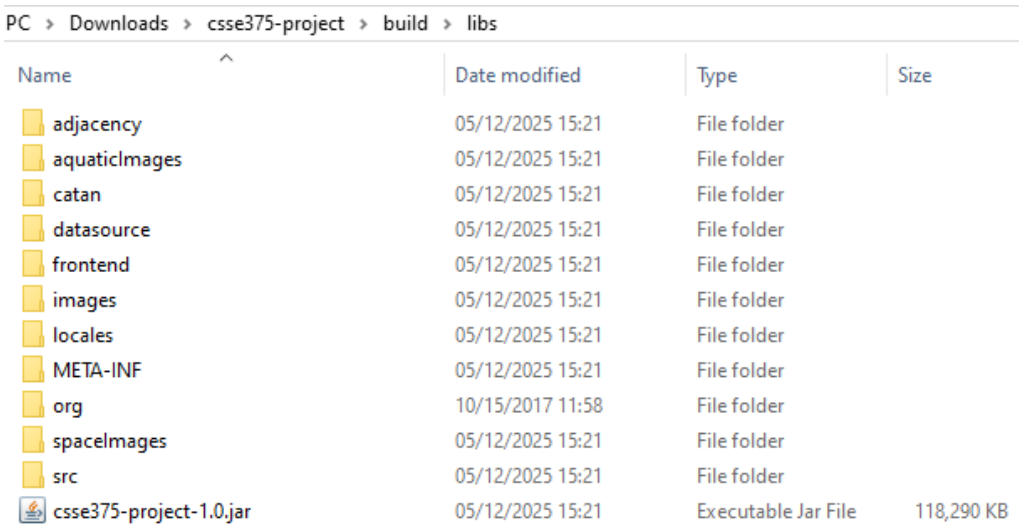
Deployment

This project is deployed by building a JAR file with Gradle and uploading it to the GitHub Releases page. Since the game depends on images, we release a zip file with those images and the JAR.

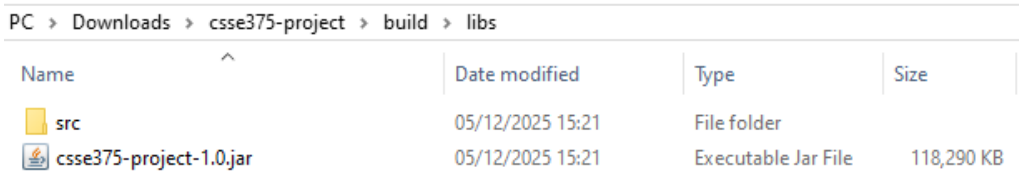
The deployment steps are outlined below.

1. Run the `build/jar` Gradle build task (see Running a Build Task)
2. The JAR file will be created in `csse375-project/build/libs` and called `csse375-project-1.0.jar`

3. In the command line, navigate to `csse375-project/build/libs`
4. Extract the files from the JAR by running `jar xf csse375-project-1.0.jar`. The `csse375-project/build/libs` directory should now contain

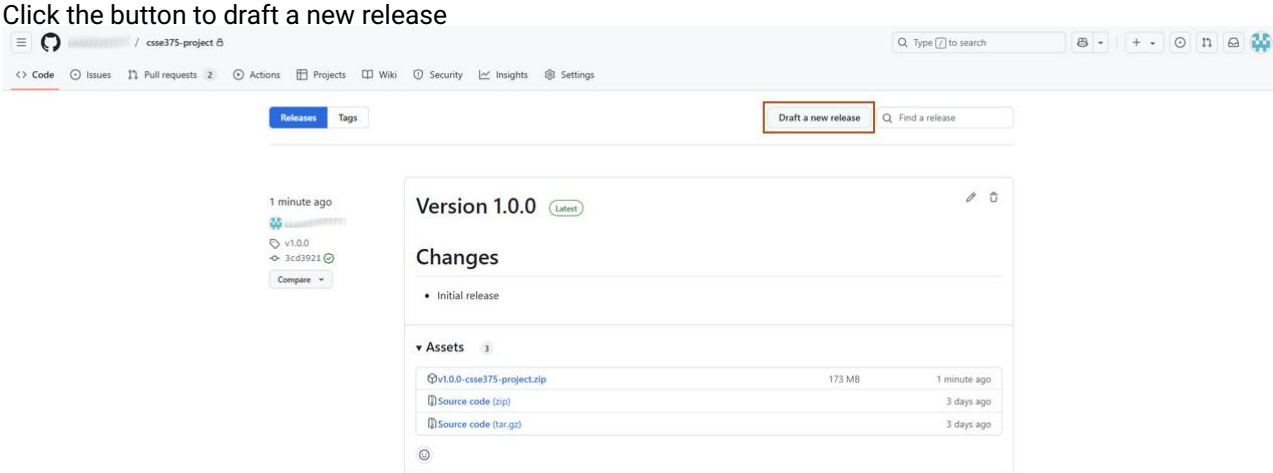


5. Delete all the directories except `src`. The `csse375-project/build/libs` directory should now contain

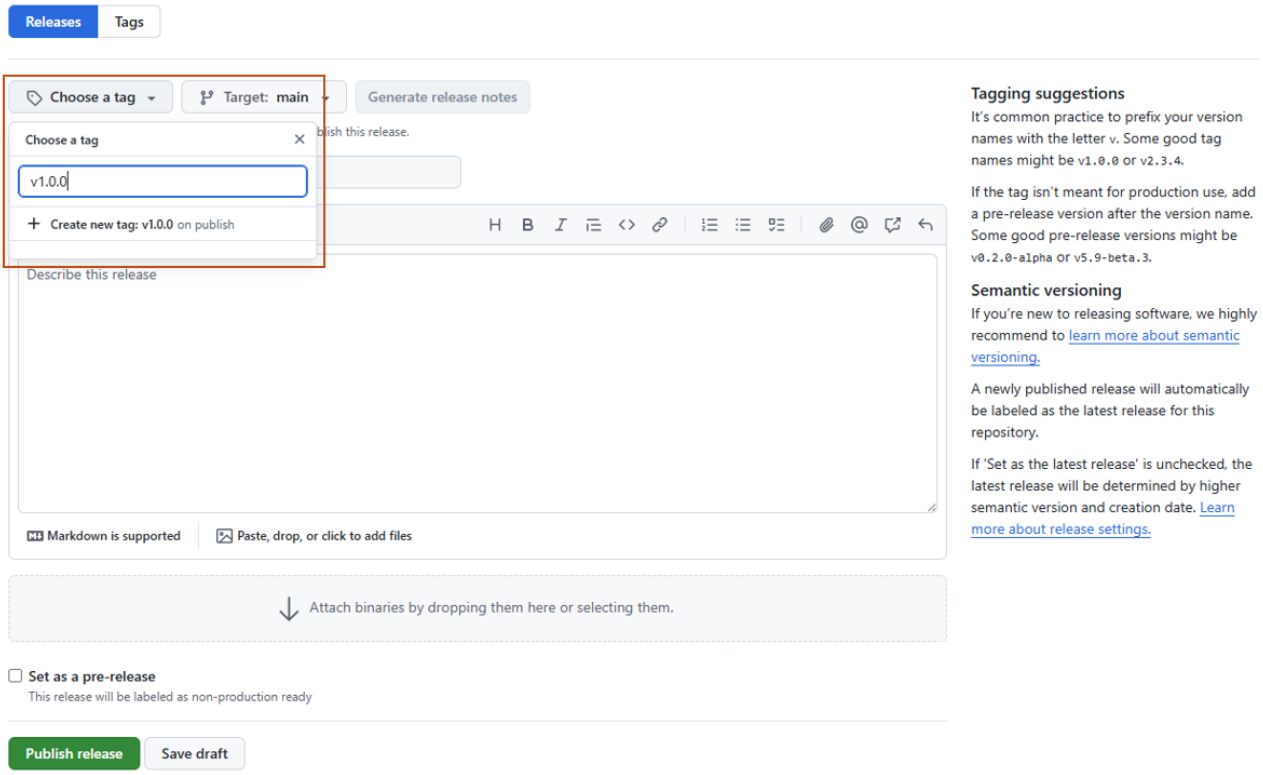


6. Rename the JAR to `v<version>-csse375-project.jar`, where `<version>` is the version number (see Version Numbering)
7. Zip
   - `src` directory
   - `v<version>-csse375-project.jar`
8. Rename the zip file to `v<version>-csse375-project`
9. Go to https://github.com/summit2021237/csse375-project/releases/

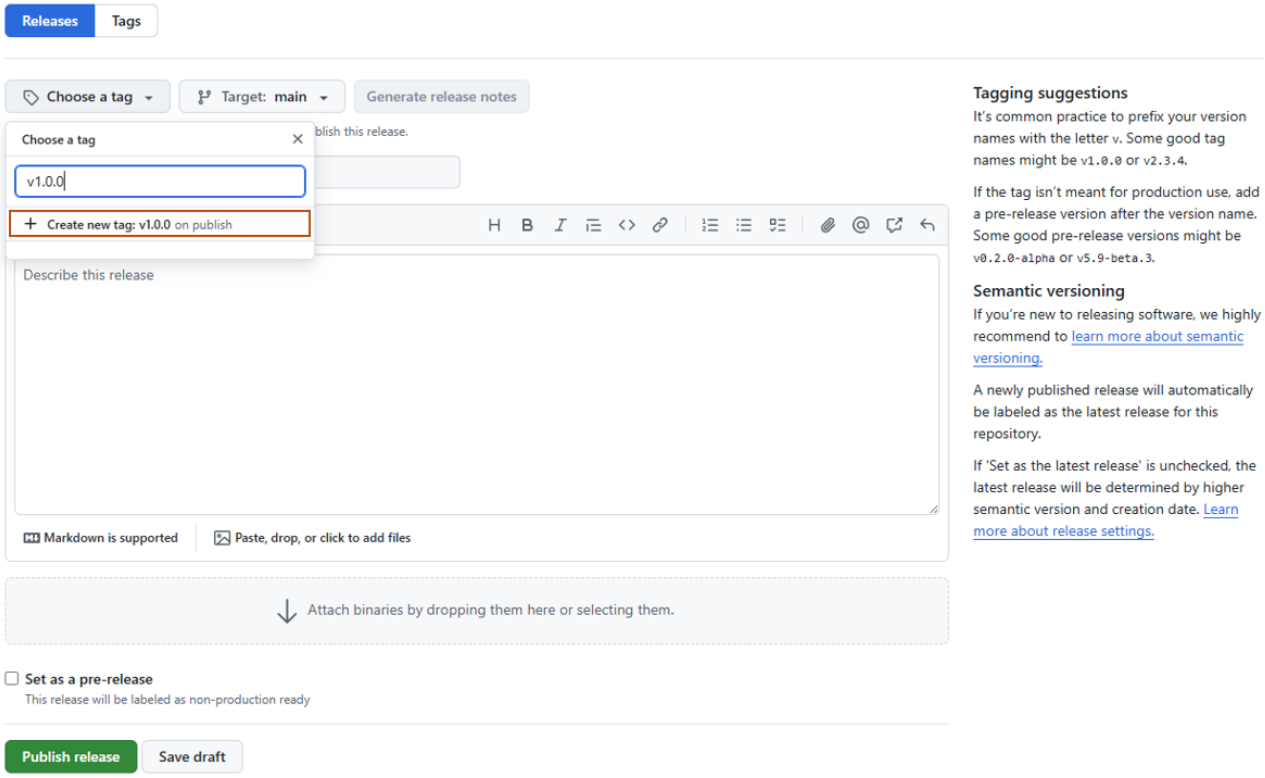
10. Click the button to draft a new release



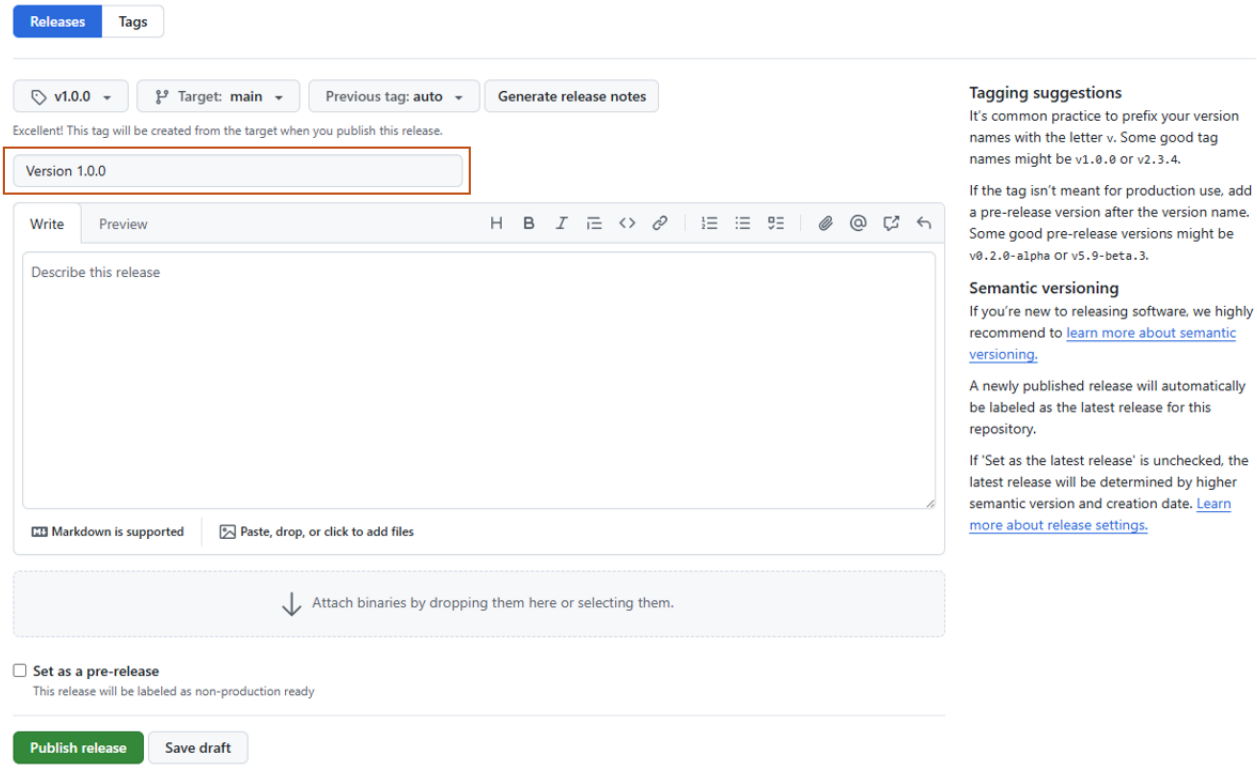
11. Enter a name for a new tag for the release version



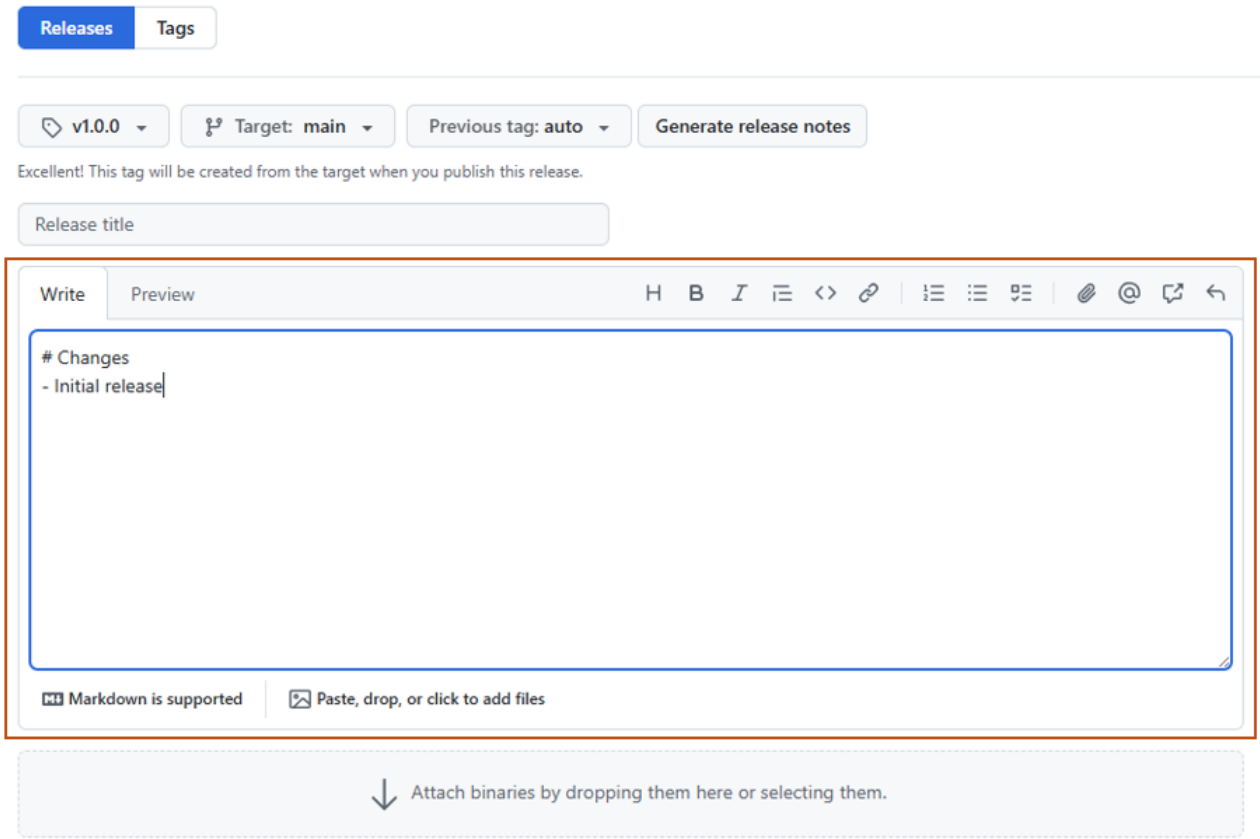
12. Select the option to create the new tag on publish



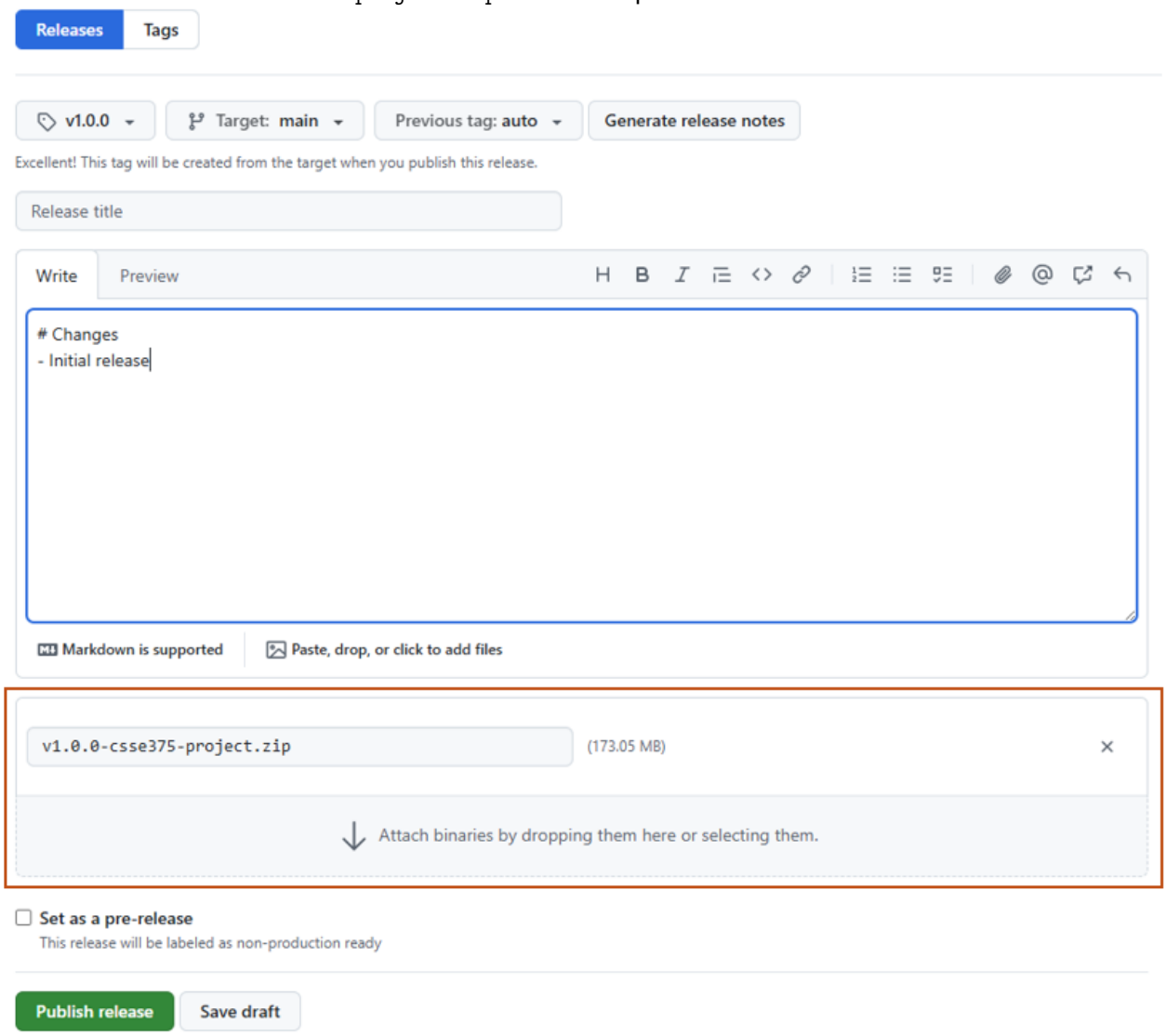
13. Set the release title based on the version number



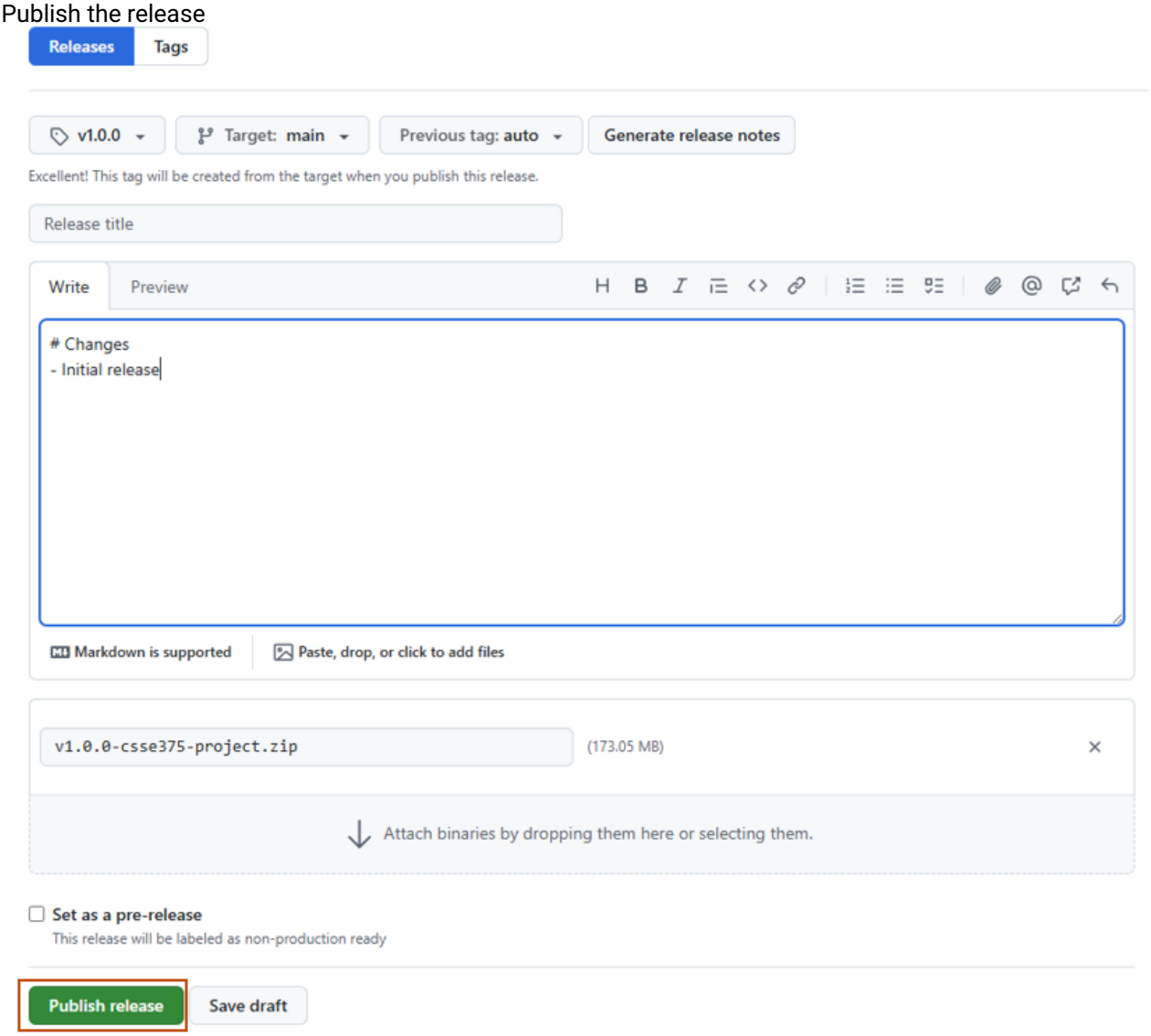
14. Describe changes made in the release under a change heading



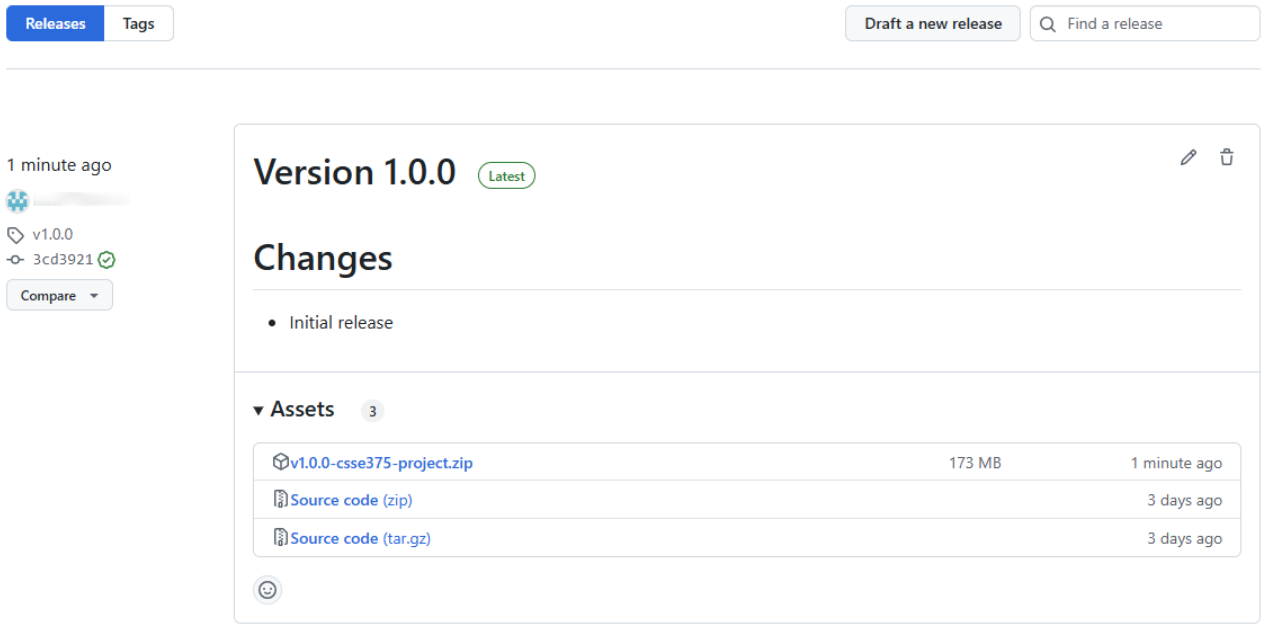
15. Attach the `v<version>-csse375-project.zip` file from step 8



16. Publish the release



17. The Releases page should now have the new release



Release Process

(Written by someone else)

Planned Changes

(Written by someone else)

Developer Troubleshooting

(Written by someone else)